



# 力扣学习

## Elegant $\text{\LaTeX}$ 经典之作

作者: Ethan Deng & Liam Huang

组织: Elegant $\text{\LaTeX}$  Program

时间: April 9, 2022

版本: 4.3

自定义: 信息



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

# 目录

|                         |           |
|-------------------------|-----------|
| <b>第 1 章 数据库基础语法</b>    | <b>1</b>  |
| 1.1 计算字符 . . . . .      | 2         |
| 1.2 函数 . . . . .        | 4         |
| 1.3 联结表 13/22 . . . . . | 8         |
| 1.4 union . . . . .     | 10        |
| <b>第 2 章 练习</b>         | <b>13</b> |
| 2.1 子查询 . . . . .       | 13        |
| 2.2 JOIN . . . . .      | 14        |

# 第 1 章 数据库基础语法

SQL 是 Structured Query Language 的缩写，中文翻译为“结构化查询语言”

## 定义 1.1 (SELECT 语句)

在 SQL 中，**SELECT** 语句是用于从数据库表中检索数据的核心指令。其基本语法结构由以下两个核心部分组成：

- 选什么 (**What to select**)：紧跟在 **SELECT** 关键字之后，用于指定需要查询的列名（字段）。
- 从哪里选 (**From where to select**)：紧跟在 **FROM** 关键字之后，用于指定数据所在的表名。

## 东邪的 tip 1.1

如果要选择多个关键字每一个关键字都要用，隔开。如果需要选择表中的所有列，可以使用通配符 **\***。

## 东邪的 tip 1.2

如果要选择名字不重复的就用 **distinct** **distinct** 是形容词所以放在要选什么的前面。**distinct** 对他后面的都有效而不是只是对他后面一个位置的有效。必须紧紧跟着 **select**

## 东邪的 tip 1.3

不同的系统选择前五语法不一样，甲骨文的是 **where rownum** 小于等于 5。如果想从要 6 到 10 行就 **limit 5 offset 5** **offset** 是偏离的意思也就偏离 5 的定语是从 1 偏离 5 也就是从第 6 个开始，他前面的就是还需要多少行。1 到 10 有 10 个数，1 到 5 有五个数所以 5-1 需要再加 1

 **笔记** 1. 物理位置 vs. 逻辑索引在 SQL 的底层逻辑里，它不把第一行看作“1”，而是看作距离起点的偏移量 (**Offset**) 为 0 的位置。2. 为什么你的“偏离 5”等于“第 6 行”？这就是你刚才理解最神的地方：如果你想从第 6 行开始看，你的 **Offset** (偏离量) 必须是 5。因为：0(起点) + 5(偏离) = 5(索引)。而索引为 5 的那一行，在人类数数时正好是第 6 个。

## 东邪的 tip 1.4

三种注释—行内注释，整行注释 **/\* \*/**

## 定义 1.2 (排序)

**order by** 必须是最后的字句，

核心逻辑拆解

数据隔离：**SELECT** 决定了最后要把哪些“列”展示在屏幕上给用户看 [cite: 5, 2026-03-07]。

排序依据：**ORDER BY** 决定了数据库在后台检索数据时，用哪根“轴”来进行对比排序 [cite: 5, 2026-03-07]。

合法性：只要这一列存在于你 **FROM** 后面的那张表（或关联表）里，数据库就能抓到它来进行排序，即便你并没有要求显示它

 **笔记** 提示：通过非选择列进行排序

通常，**ORDER BY** 子句中使用的列将是为显示而选择的列。但是，实际上并不一定要这样，用非检索的列排序数据是完全合法的。

## 多列排序示例

```
/* 下面的代码检索 3 个列，并按其中两个列对结果进行排序——首先按价格，然后按名称排序。 */  
SELECT prod_id, prod_price, prod_name
```

```
FROM Products
ORDER BY prod_price, prod_name;
```

Listing 1.1: 使用列位置排序

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY 2, 3;
```

这个就是先按照 price 排序再按照 name 排序也就是先价格再首字母。也就是按照 name 的首字母排序。用这个方法局限的是只能用 select 里面的列作为排序的依据。desc 可以按照降序排列他的作用只在他前面的东西。

**定义 1.3 (where 用来过滤数据)**

where 后面跟着限定条件比如 price=3.5; 记住 order by 得写在他的后面

**定义 1.4 (between)**

between 和 where 要一起用 where between 爱你的、

**定义 1.5 (or+and+In)**

where+and 可以进行高级筛选可以一个 where——多个 and, or 是或。and 的优先级比 or 高如果需要先进行 or 判断必须加括号。有 and 有 or 用括号。In (,) 需要有括号每两个不同的用, 隔开。也是 where in。not 否定跟着后面的条件在条件复杂的时候尤为有效

**东邪的 tip 1.5**

通配符用来匹配值一部分的特殊字符, 比如说含有 bean bag, like 得配合着 where 用, 立刻后面跟着 'fish'F

 **笔记** 下划线 (\_) vs 百分号 (%)

- % 是“贪婪”的: 它能匹配 0 个、1 个或无数个字符。比如 '% inch' 能匹配 8 inch, 也能匹配 12 inch。
- \_ 是“死板”的: 一个下划线就代表一个占位符。

看图 3 中的代码: LIKE '\_\_ inch teddy bear' (注意这里是两个下划线)。

 **作业**

1. 为什么 12 inch 和 18 inch 能匹配?  
因为 12 是两个字符, 刚好填满了那两个下划线的坑。
2. 为什么 8 inch 匹配不到?  
因为 8 只有一个字符。如果你用了两个下划线 \_\_, SQL 就会在心里数数: “一、二... 诶? 8 后面没字符了, 但我这里要求必须有两个字符才能过关”, 于是就把它剔除了。

**东邪的 tip 1.6 (and 后面还需要写一遍判断主体 prod\_descname)**

```
select prod_name, prod_desc from Products where prod_desc like "
```

## 1.1 计算字符

**输入**

```
SELECT prod_id,
       quantity,
       item_price,
       quantity*item_price AS expanded_price
```

```
FROM OrderItems
WHERE order_num = 20008;
```

输出

| prod_id | quantity | item_price | expanded_price |
|---------|----------|------------|----------------|
| RGAN01  | 5        | 4.9900     | 24.9500        |
| BR03    | 5        | 11.9900    | 59.9500        |
| BNBG01  | 10       | 3.4900     | 34.9000        |
| BNBG02  | 10       | 3.4900     | 34.9000        |
| BNBG03  | 10       | 3.4900     | 34.9000        |

#### 东邪的 tip 1.7

如何进行加减乘除，把运算完的起一个名字即可然后就可以搜索了

section 计算字符 计算字符涉及到了合并列等等操作要紧跟着 select

输入：

```
SELECT vend_name + ' (' + vend_country + ')'
FROM Vendors
ORDER BY vend_name;
```

输出：

|                 |            |
|-----------------|------------|
| Bear Emporium   | (USA )     |
| Bears R Us      | (USA )     |
| Doll House Inc. | (USA )     |
| Fun and Games   | (England ) |
| Furball Inc.    | (USA )     |
| Jouets et ours  | (France )  |

#### 东邪的 tip 1.8

+ 可以用来拼接' ' 是一个整体

输入

```
SELECT RTRIM(vend_name) + ' (' + RTRIM(vend_country) + ')'
FROM Vendors
ORDER BY vend_name;
```

输出

```
Bear Emporium (USA)
Bears R Us (USA)
Doll House Inc. (USA)
Fun and Games (England)
Furball Inc. (USA)
Jouets et ours (France)
```

#### 东邪的 tip 1.9

rtrim 函数可以去掉所有右边空格，比如这里去掉的是 `vend_nameRT + rimLfT`

输入



```
SELECT RTRIM(vend_name) + '_( ' + RTRIM(vend_country) + ' )'
       AS vend_title
FROM Vendors
ORDER BY vend_name;
```

**输出**

vend\_title

---

Bear Emporium (USA)  
 Bears R Us (USA)  
 Doll House Inc. (USA)  
 Fun and Games (England)  
 Furball Inc. (USA)  
 Jouets et ours (France)

**东邪的 tip 1.10**

as 赋予了新的列一个名字也就是合成的这个列也有名字了可以 select 了

## 1.2 函数

**东邪的 tip 1.11**

upper 函数会让所有字母都大写括号里面就是大写的对象

**表 1.1:** 常用的文本处理函数

| 函数                                  | 说明                  |
|-------------------------------------|---------------------|
| LEFT() (或使用子字符串函数)                  | 返回字符串左边的字符          |
| LENGTH() (或使用 DATALENGTH() 或 LEN()) | 返回字符串的长度            |
| LOWER()                             | 将字符串转换为小写           |
| LTRIM()                             | 去掉字符串左边的空格          |
| RIGHT() (或使用子字符串函数)                 | 返回字符串右边的字符          |
| RTRIM()                             | 去掉字符串右边的空格          |
| SUBSTR() 或 SUBSTRING()              | 提取字符串的组成部分 (见表 8-1) |
| SOUNDEX()                           | 返回字符串的 SOUNDEX 值    |
| UPPER()                             | 将字符串转换为大写           |

**东邪的 tip 1.12**

soundex() 方便寻找拼写错误的, 但是发生想的比如如果录入的时候 Michel 打错了大成 Michelle 了那么就可以在 where 后面加上判断条件 where soundex(vend\_name)=soundex(Michel)

字母拼起来不能用——要用 concat

**东邪的 tip 1.13**

```
select cust_id,cust_name, upper(CONCAT (left (cust_contact,2),left(cust_city,3))) as
user_logn from Customers
```

```
SELECT AVG(prod_price) AS avg_price
FROM Products;
```

输出：

```
avg_price
-----
6.823333
```

#### 东邪的 tip 1.14

记住每一次运算完都要给新结果一个名字。avg 只能运算单列会忽略 null 值，

- 使用 COUNT(\*) 对表中行的数目进行计数，不管表列中包含的是空值 (NULL) 还是非空值。
- 使用 COUNT(column) 对特定列中具有值的行进行计数，忽略 NULL 值。

#### 东邪的 tip 1.15

group by 是和 where 配合操作目的是对一类目标形成筛选和操作。group by 在 where 后面在 order by 前面

#### 笔记 GROUP BY 的使用注意事项：

- GROUP BY 子句中列出的每一项都必须是检索列或有效的表达式，但不能是聚集函数。也就是说，GROUP BY 后面应当写“具体的分组依据”，而不能写 COUNT(\*)、SUM(price) 这类分组之后才得到的结果。
- 如果在 SELECT 中使用了表达式，例如 YEAR(order\_date)，那么在 GROUP BY 中也必须写相同的表达式，而不能只按原列分组。
- 在很多 SQL 实现中，GROUP BY 中不能直接使用 SELECT 子句里定义的别名，因此通常应写原表达式，而不要写别名。
- 大多数 SQL 实现不允许对长度可变的大字段进行分组，例如文本型字段或备注型字段。

#### 笔记 WHERE 与 HAVING 的区别：

- WHERE 用于在分组之前对原始记录进行筛选，决定哪些行能够进入分组，因此 WHERE 是“管行”的。
- HAVING 用于在分组之后对分组结果进行筛选，决定哪些分组能够保留下来，因此 HAVING 是“管组”的。
- 一般来说，WHERE 中不能使用聚集函数，而 HAVING 中可以使用聚集函数。

**例题 1.1** 下面的 SQL 语句先筛选工资大于 5000 的员工，再按部门编号分组，最后只保留人数不少于 3 的部门：

```
SELECT dept_id, COUNT(*)
FROM employee
WHERE salary > 5000
GROUP BY dept_id
HAVING COUNT(*) >= 3;
```

#### 笔记 上述语句的执行过程可以理解为：

- 先由 WHERE salary > 5000 筛选出工资大于 5000 的员工记录；
- 再按照 dept\_id 对这些记录进行分组；
- 最后由 HAVING COUNT(\*) >= 3 保留员工人数不少于 3 的那些部门分组。

#### 东邪的 tip 1.16

聚集函数如果和 GROUP BY 一起出现，默认就是“对每个组聚集”。所以聚类函数 (aggregate function) count all 默认就是 count 刚分好的组。

 **笔记** 虽然 SELECT 子句写在最前面，但 SQL 查询在逻辑上并不是先执行 SELECT。一般可理解为按如下顺序执行：

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

因此，COUNT(\*) 虽然写在 SELECT 中，但如果语句里有 GROUP BY，它实际统计的是每个分组中的行数。

**表 1.2:** ORDER BY 与 GROUP BY 的区别

| ORDER BY               | GROUP BY                      |
|------------------------|-------------------------------|
| 对产生的输出排序               | 对行分组，但输出可能不是分组的顺序             |
| 任意列都可以使用（甚至非选择的列也可以使用） | 只可能使用选择列或表达式列，而且必须使用每个选择列表表达式 |
| 不一定需要                  | 如果与聚集函数一起使用列（或表达式），则必须使用      |

#### 东邪的 tip 1.17

用 group by 不能指望他能排好

**表 1.3:** ORDER BY 与 GROUP BY 的区别

| ORDER BY               | GROUP BY                      |
|------------------------|-------------------------------|
| 对产生的输出排序               | 对行分组，但输出可能不是分组的顺序             |
| 任意列都可以使用（甚至非选择的列也可以使用） | 只可能使用选择列或表达式列，而且必须使用每个选择列表表达式 |
| 不一定需要                  | 如果与聚集函数一起使用列（或表达式），则必须使用      |

**例题 1.2** `select order_num, count(*) as group_line from OrderItems group by order_num group by count`

**例题 1.3** 下面是一条 SQL 查询语句，用于查询订购了产品 RGAN01 的顾客编号：

```
SELECT cust_id
FROM Orders
WHERE order_num IN (SELECT order_num
                     FROM OrderItems
                     WHERE prod_id = 'RGAN01');
```

其输出结果为：

```
cust_id
-----
1000000004
1000000005
```

#### 东邪的 tip 1.18

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY 这里是从两个不同的表里面 select 并且把第一个的结果当成

**例题 1.4** 下面这个写法是一个典型错误：

```
SELECT cust_email
```



```

FROM Customers
WHERE cust_id IN (
    SELECT order_date, cust_id
    FROM Orders
    WHERE order_num IN (
        SELECT order_num
        FROM OrderItems
        WHERE prod_id = 'BR01'
    )
);

```

错误原因在于：外层条件是

cust\_id IN (子查询)

这里左边只有一个值 cust\_id，因此括号中的子查询必须只返回一列，并且这一列应当能够与 cust\_id 进行比较。

但是该子查询返回了两列：

order\_date, cust\_id

这就与 IN 的要求不匹配，因此会报错。

正确写法应为：

```

SELECT cust_email
FROM Customers
WHERE cust_id IN (
    SELECT cust_id
    FROM Orders
    WHERE order_num IN (
        SELECT order_num
        FROM OrderItems
        WHERE prod_id = 'BR01'
    )
);

```

也就是说，当写成

某列 IN (子查询)

时，子查询通常只能返回一列；如果返回多列，就会出现这种典型错误。



**笔记** SQL 里子查询常见有三种位置：1. 写在 WHERE 后面：拿来筛选 2. 写在 FROM 后面：拿来当一张临时表 3. 写在 SELECT 后面：拿来给每一行补一个值 select 后面可以是表中没有的也就是处理过的新添加的

**例题 1.5** 下面的查询用于返回每个顾客的顾客 ID 以及其已订购商品的总金额，并按总金额从大到小排序：

```

SELECT cust_id,
    (SELECT SUM(quantity * item_price)
     FROM OrderItems
     WHERE order_num IN (SELECT order_num
                        FROM Orders
                        WHERE Orders.cust_id = Customers.cust_id)) AS total_ordered
FROM Customers
ORDER BY total_ordered DESC;

```

**东邪的 tip 1.19**

先把最终要输出的 `cust_id` 和 `as total_orders` 写在 `select` 后，这里子句明显是作为计算字段，因为要添加表里面原来没有的东西。所以 `select` 肯定在 `from` 前面。一共需要两步，我们先算总价格，这个是容易想的；但是算总价格这步不能建立起 `orders` 和 `customers` 的相同 `id` 的映射，因为 `where in` 这种筛选只能里面有一个 `select`，所以还需要一个筛选的 `select`。最后有一个 `as` 的名字就行了，`sum` 的时候就不用有名字。

**东邪的 tip 1.20 (一个顾客可以下好几个订单，每一个订单可以买好几个东西)**

建立两个表的联系就是 `where A in (select A from ...)`，`A` 的位置只能是一个。这里如果要计算 `sum`，就需要知道 `sum` 的是那个名字的所有的数量 \* 价格，所以我们必须要筛选相同 `order_num` 的东西。写出标题话我们就可以知道，算订单总价紧跟着的必须是订单号，筛选订单号紧跟着的必须是顾客 `id`。可以从侧栏点当前输入，还可以检查这个表里面到底有没有。

## 1.3 联结表 13/22

**东邪的 tip 1.21 (意义)**

把信息分解成不同的表，通过共同的值关联。• 供应商表：

供应商表: `vend_id, vend_name`

商品表: `prod_id, vend_id, prod_name`

```
SELECT vend_name, prod_name, prod_price
FROM Vendors
INNER JOIN Products ON Vendors.vend_id = Products.vend_id;
```



**笔记** 此语句中的 `SELECT` 与前面的 `SELECT` 语句相同，但 `FROM` 子句不同。这里，两个表之间的关系是以 `INNER JOIN` 指定的部分 `FROM` 子句。在使用这种语法时，联结条件用特定的 `ON` 子句而不是 `WHERE` 子句给出。传递给 `ON` 的实际条件与传递给 `WHERE` 的相同。

**东邪的 tip 1.22**

`inner join A on B` 很清晰的会知道是 `A` 和 `B` 做联结 • `WHERE`：逻辑上像先全组合再筛 • `INNER JOIN`：逻辑上更像“按条件配对”

**例题 1.6** 在多表联结中，若某个字段名在多个表中都出现过，就需要写成“表名.字段名”的形式，以消除歧义。例如这里的 `order_num` 在 `Orders` 表和 `OrderItems` 表中都可能出现，因此应写成 `Orders.order_num`。

另外，若在 `SELECT` 中使用了聚合函数 `sum(...)`，那么凡是同时出现但没有被聚合的字段，都必须写在 `GROUP BY` 中。所以 `cust_name` 和 `Orders.order_num` 都要分组。

对应的 SQL 语句为：

```
select cust_name, Orders.order_num,
       sum(quantity*item_price) as ordertotal
from Customers
inner join Orders
on Customers.cust_id = Orders.cust_id
inner join OrderItems
on OrderItems.order_num = Orders.order_num
group by cust_name, Orders.order_num;
```

**定义 1.6 (等联结)**

若两个表进行联结时，联结条件是两个字段之间的相等关系，即使用等号 = 来建立匹配关系，则这种联结称为等联结。

例如：Customers.cust\_id = Orders.cust\_id 和 Orders.order\_num = OrderItems.order\_num 都是等联结条件。

**东邪的 tip 1.23**

有 group by 再想筛选 group by 也就是分组后产生的结果就要用 having, where 只能在 group by 前面筛选分组前的内容

**例题 1.7** 下面语句用于查询订单总额不少于 1000 的顾客编号和订单编号：

```
select Orders.cust_id, OrderItems.order_num
from Orders
inner join OrderItems
on Orders.order_num = OrderItems.order_num
group by Orders.cust_id, OrderItems.order_num
having sum(item_price * quantity) >= 1000;
```

常见错误如下：

1. 将 OrderItems 误写为 OrdersItems。
2. 将聚合条件写在 where 中，而不是写在 having 中。
3. select 中出现的非聚合列没有全部写入 group by。
4. 多表查询时，对重名字段没有写成 表名. 字段名，从而产生歧义。

```
SELECT cust_name, cust_contact
FROM Customers AS C, Orders AS O, OrderItems AS OI
WHERE C.cust_id = O.cust_id
      AND OI.order_num = O.order_num
      AND prod_id = 'RGAN01';
```

这样就可以缩短表头的书写。

 **笔记** [自联结] 可以，因为这里不是两个不同的 Customers 表，而是同一张表起了两个别名：

- Customers AS c1
- Customers AS c2

相当于把同一张表“当成两份来看”。

这样做是为了让表自己和自己比较，这种连接称为自联结（self join）。

这句查询语句的含义是：

- c2.cust\_contact = 'Jim Jones': 先找到联系人是 Jim Jones 的那一行；
- 再利用 c1.cust\_name = c2.cust\_name，找到与他姓名相同的其他顾客。

所以，c1 和 c2 只是同一张表在查询中扮演的两个不同角色，并不冲突。

 **笔记**

1. INNER JOIN 只保留能匹配上的行。
2. OUTER JOIN 除了匹配上的行，还会保留没匹配上的一边或两边。也就是会给没有的补一个 null，必须前面有 right 或者 left 说明他到底补那边。还有 full 就是两边都补  
customer left outer join 就是左侧也就 customer 全部填满 select 是倒数第二优先级运行只比 order by 高一级

## 1.4 union

### 东邪的 tip 1.24

union 可以连接两个 select 但是两个的 select 内容必须格式相同。union 自动删除重复的项。如果想要重复的项就 union all，如果排序 order by 必须在最后一个 select 的最后面

**例题 1.8** 在 SQL 中，UNION 要求两个 SELECT 的结果结构兼容，即列数相同，且对应位置的数据类型可以匹配；它并不要求两边查询的是同一种含义的数据。

例如下面是可以的：

```
SELECT prod_name
FROM Products
UNION
SELECT cust_name
FROM Customers;
```

因为两边都只返回 1 列，且通常都是字符型。这个的效果就是两个合成一列  
但下面不可以：

```
SELECT prod_name, prod_id
FROM Products
UNION
SELECT cust_name
FROM Customers;
```

因为左边返回 2 列，而右边只返回 1 列，列数不一致。

### 东邪的 tip 1.25

insert into 要插的表加 (表的表头) value () 如果没有的值要用 null 填充。insert select 用法自动从新表 select 东西合并到老表里面

creat 新表 select

### 东邪的 tip 1.26 (update)

要更新的表的名字，要更新的内容，还有用于更新具体地方的条件。update+set。update 可以用子查询

```
UPDATE 表名
SET 列名 = 新值
WHERE 条件;
```

### 东邪的 tip 1.27 (delete)

可以删除特定的行和删除所有的行，他是删除行删除不了列

```
DELETE FROM 表名
WHERE 条件;
```

1. 第一个问题是最开始的语句写法不对，SET 后面不能只写 UPPER(vend\_state)，而要写成

```
vend_state = UPPER(vend_state)
```

2. 第二个问题是没有写 WHERE 条件，在 MySQL 的安全更新模式下，直接更新整张表会被禁止。
3. 第三个问题是后来虽然加了 WHERE vend\_state IS NOT NULL，但 vend\_state 不是键列（key column），所以在安全模式下仍然不能执行。

### 东邪的 tip 1.28

create table 就是后面跟着表名字然后（表头 + 数据类型 + 能否是 null 值，可以用 default 规定默认值，drop table 就能删除整个表）

```
CREATE TABLE 表名
(
    列名1 数据类型 约束,
    列名2 数据类型 DEFAULT 默认值
);
```

```
DROP TABLE 表名;
```

```
ALTER TABLE 表名
ADD 列名 数据类型;
```

```
ALTER TABLE 表名
DROP COLUMN 列名;
```

### 东邪的 tip 1.29

所以 view 就相当于一个加工好的东西，如果需要建新的或者覆盖只能删除了在建

假设不想让别人看到 prod\_desc，可以建一个视图只暴露三列：

```
CREATE VIEW ProductsPublic AS
SELECT prod_id, prod_name, prod_price
FROM Products;
```

以后查数据只需：

```
SELECT * FROM ProductsPublic;
```

prod\_desc 对用户完全不可见。若需简化重复查询，例如常用“价格超过 10 的产品”，可建视图：

```
CREATE VIEW ExpensiveProducts AS
SELECT prod_name, prod_price
FROM Products
WHERE prod_price > 10;
```

以后直接查询该视图即可，无需每次重写 WHERE 条件：

```
SELECT * FROM ExpensiveProducts;
```

**例题 1.9JOIN 视图** 创建仅含下过单顾客的视图：

```
CREATE VIEW CustomersWithOrders AS
SELECT Customers.*, Orders.order_num, Orders.order_date
FROM Customers
JOIN Orders ON Customers.cust_id = Orders.cust_id;
```

常见错误：JOIN 后漏写 ON；用 SELECT \* 导致 cust\_id 重复 (Error 1060)；重建视图前未执行 DROP VIEW (Error 1050)。注意 INNER JOIN 已自动过滤未下单顾客，无需额外 WHERE。

**例题 1.10 计算列视图** 创建含总价计算列的视图：

```
CREATE VIEW OrderItemsExpanded AS
SELECT order_num, prod_id, quantity, item_price,
       quantity*item_price AS expanded_price
FROM OrderItems;
```

常见错误：在视图定义中写 ORDER BY，MySQL 不保证其生效，应在查询视图时再加排序。视图可直接存储计算列，避免每次查询重复计算。



## 第2章 练习

### 2.1 子查询

子查询就是创建一个池子以前要写 a 在 1, 2, 3 里面选但是用另一个表表里就是 1, 2, 3。就用 select from 这个表直接返回 1, 2, 3。当然作为比较条件也可以是大余小余找第二大的去掉最大的就行。

```
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employee
WHERE Salary < (SELECT MAX(Salary) FROM Employee);
```

#### 定义 2.1 (SQL 自定义函数)

自定义函数 (User-Defined Function) 是用户在数据库中创建的可复用逻辑单元，语法为：

```
CREATE FUNCTION 函数名(参数名 参数类型, ...) RETURNS 返回类型
BEGIN
    -- 函数体
    RETURN 返回值;
END
```

与存储过程的区别在于：函数必须有返回值，且可以直接嵌套在 SELECT 语句中使用。



#### 定义 2.2 (SQL 循环)

循环只能写在存储过程或自定义函数内部，MySQL 提供三种循环结构：

```
-- 1. LOOP: 手动控制退出
loop1: LOOP
    IF 条件 THEN LEAVE loop1; END IF;
END LOOP;

-- 2. WHILE: 先判断条件再执行
WHILE 条件 DO
    -- 执行内容
END WHILE;

-- 3. REPEAT: 先执行再判断，至少执行一次
REPEAT
    -- 执行内容
UNTIL 条件 END REPEAT;
```



#### 例题 2.1 \*[177 第 n 高薪水]

```
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
    SET N = N - 1;
    RETURN (
        SELECT DISTINCT Salary
        FROM Employee
```

```
ORDER BY Salary DESC  
LIMIT 1 OFFSET N  
);  
END
```

## 2.2 JOIN